

Vectera PMR Generator: Write-up

Overview of what was done

I built a tool that turns quarterly documents into a Performance Measurement Report. It reads the flash spreadsheet, prior PMR, manager PDFs, market deck, IC logs and allocation history. I wanted to automate repetitive work while keeping a reviewer in control.

Where the model stops and code starts

I kept numbers, rankings and decisions in plain code that gives the same answer every time. Python finds the files, checks the numbers match, calculates dollar contributions, selects featured funds, filters IC activity and tests compliance. Returns come from the rows named by SPEC, not from model calculations. Figure and role sentences use templates.

The model reads chart images and writes connecting prose for funds, new commitments and the market update. The prose writer gets number placeholders rather than real figures. Every sentence must cite a source passage. Code fills in the source figures and rejects sentences that add, drop, change or borrow numbers. A second model pass checks the wording against the cited quote. If writing fails these checks, I use an attributed source excerpt instead.

How I know it's right

The supplied quarter passes 58 checks that the numbers match. Tests check the four featured funds, \$456.8M committed or approved across 14 positions, recent activity and compliance results. Expected sample values live in tests, not the reporting code. Other tests cover missing values, shuffled rows, fabricated quotes and changed approval files.

I tested repeatability in separate processes with empty caches. The comparisons cover figures, chart readings, fund selection, roles, compliance, section structure and the source context behind prose numbers. Wording can vary between fresh model calls. CI is configured for tests and no-key repeatability on Windows and macOS with Python 3.12. The fresh-install script unpacks the submission, installs pinned dependencies and runs tests and generation. The recorded local pass is on macOS.

Where a human steps in

The review page has Report, Charts and Issues views. A reviewer can inspect source links, compare chart readings with images and submit corrections with a reason. A chart correction creates a new draft, replaces the old market prose and requires another review. It cannot quietly become an approved report.

Warnings ask for attention without stopping a draft: Meridian has no manager report, Meridian's fees are negative, and the allocation workbook has no client column. Blockers prevent release: a portfolio total that does not add up, a missing holding NAV needed for ranking, or an unresolved approved commitment amount. Missing values stay unavailable rather than becoming zeros or partial totals.

Approval requires confirmed charts, no blockers, a named reviewer and an acknowledgment. File hashes tie approval to the reviewed draft and final PDF. Changed source files, code, draft data or displayed PDF stop approval.

Next quarter and format changes

For the next quarter, I replace the input package and provide the client and quarter. Files are selected by content, and tables by labels and headers. The flash supplies the client name and benchmark; the prior PMR supplies policy limits, managers and disclosures. IC logs are read across years so earlier approvals are not lost.

The 1Q26 rehearsal changes the client, renames files, moves rows, adds and removes funds, and carries an approval into a later withdrawal. It runs without code edits. The prior PMR supplies section order and typography; unknown sections are flagged. A new financial measure still needs code. I retain current flash columns in portrait appendix tables, including horizons absent from the prior sample.

Briefly

Architecture: I used one pipeline for discovery, calculations, evidence, writing and rendering. The browser workspace and command line run the same pipeline. I left out a database and Docker to keep setup small.

Traceability: manifest.json connects report text and table cells to where they came from. evidence.json records facts, formulas and inputs. A reviewer can follow a figure back to its sheet and cell, PDF page or slide.

Messy inputs and name matching: explicit rules handle punctuation, legal suffixes and Roman or Arabic numerals. Close matches are suggestions, not automatic merges. A matched report with unreadable commentary is not described as missing.

Models and tools: I used pinned GPT-4.1 for prose, GPT-4.1-mini for charts, PyMuPDF for PDFs and ReportLab for output. Calls use temperature zero and a local cache. With no API key, code still produces figures and attributed excerpts, but unavailable chart readings block approval.

Anthropic also completed two fresh Sonnet 4.6 runs with matching checked output. However, it called an unlabeled chart's readings printed labels, although the values matched. I kept the OpenAI submission because that mistake removed the approximation markers.

One judgment call: since-inception IRR uses the "Vectera Initiated Investments" row, displayed as 9.3%, rather than "Cascadia Portfolio" at 10.2%. SPEC names that row for the total portfolio. It also matches the prior report's 9.2% choice.

With more time, I would cross-check chart images with code-based measurement. I would add more cross-client test packages. Shared use would also need authenticated reviewer accounts.

Where SPEC and the prior PMR disagree

The sample's "Of which funded \$391.8M" is the commitment total, not the \$475.5M funded amount. I label it "Flash commitment total." I keep market indices separate from the client benchmark and include every required attribution role. I carry forward the prior report's 2001 retention sentence without resolving its conflict with older flash inception dates.